

Description of application:

Our application validates a completed 9 x 9 sudoku board that the user inputs. It reads a 9 x 9 grid and determines whether or not it follows the rules of sudoku, such that there is no repetition in each row and column, and no repetition in 3 x 3 subgrids. The application outputs either Valid! or Invalid! when completed.

Algorithms and Data Structures used and key benefits:

The program uses hash-based validation through the C++ `unordered_set` data structure to efficiently check for duplicates in rows, columns, and 3×3 subgrids. By attempting to insert each number into an `unordered_set`, the program can instantly detect duplicates, since failed insertions indicate repeated values. This approach provides average constant-time complexity $O(1)$ for insert and lookup operations. The use of sets also simplifies logic, reducing the need for nested comparison loops.

Group member roles and responsibilities:

Jason Smevog: Implemented the main function and the google document.

Yui Nguyen: Implemented the `validSudoku` function and created the algorithm we used.

Yusra Ashar: Created the input and output test cases.

Test Cases:

```
input00.txt
1  5 3 4 6 7 8 9 1 2
2  6 7 2 1 9 5 3 4 8
3  1 9 8 3 4 2 5 6 7
4  8 5 9 7 6 1 4 2 3
5  4 2 6 8 5 3 7 9 1
6  7 1 3 9 2 4 8 5 6
7  9 6 1 5 3 7 2 8 4
8  2 8 7 4 1 9 6 3 5
9  3 4 5 2 8 6 1 7 9

output00.txt
1  Valid!
2
```

```
input01.txt
1  5 3 4 6 7 8 9 1 1
2  6 7 2 1 9 5 3 4 8
3  1 9 8 3 4 2 5 6 7
4  8 5 9 7 6 1 4 2 3
5  4 2 6 8 5 3 7 9 1
6  7 1 3 9 2 4 8 5 6
7  9 6 1 5 3 7 2 8 4
8  2 8 7 4 1 9 6 3 5
9  3 4 5 2 8 6 1 7 9

output01.txt
1  Invalid!
2
```

```

input02.txt
1  5 3 4 6 7 8 9 1 2
2  6 7 2 1 9 5 3 4 8
3  5 9 8 3 4 2 5 6 7
4  8 5 9 7 6 1 4 2 3
5  4 2 6 8 5 3 7 9 1
6  7 1 3 9 2 4 8 5 6
7  9 6 1 5 3 7 2 8 4
8  2 8 7 4 1 9 6 3 5
9  3 4 5 2 8 6 1 7 9

```

```

output02.txt
1  Invalid!
2

```

```

input03.txt
1  5 3 4 6 7 8 9 1 2
2  6 7 2 1 9 5 3 4 8
3  1 9 5 3 4 2 5 6 7
4  8 5 9 7 6 1 4 2 3
5  4 2 6 8 5 3 7 9 1
6  7 1 3 9 2 4 8 5 6
7  9 6 1 5 3 7 2 8 4
8  2 8 7 4 1 9 6 3 5
9  3 4 5 2 8 6 1 7 9

```

```

output03.txt
1  Invalid!
2

```

```

input04.txt
1  5 3 0 0 7 0 0 0 0
2  6 0 0 1 9 5 0 0 0
3  0 9 8 0 0 0 0 6 0
4  8 0 0 0 6 0 0 0 3
5  4 0 0 8 0 3 0 0 1
6  7 0 0 0 2 0 0 0 6
7  0 6 0 0 0 0 2 8 0
8  0 0 0 4 1 9 0 0 5
9  0 0 0 0 8 0 0 7 9

```

```

output04.txt
1  Invalid!
2

```

Screenshot of passing tests:

```

@JSmevog1 →/workspaces/hw-group1-c-JSmevog1 (main) $ clang++ -o a.out -std=c++17 main.cpp
@JSmevog1 →/workspaces/hw-group1-c-JSmevog1 (main) $ ./a.out < input00.txt | diff - output00.txt
@JSmevog1 →/workspaces/hw-group1-c-JSmevog1 (main) $ ./a.out < input01.txt | diff - output01.txt
@JSmevog1 →/workspaces/hw-group1-c-JSmevog1 (main) $ ./a.out < input02.txt | diff - output02.txt
@JSmevog1 →/workspaces/hw-group1-c-JSmevog1 (main) $ ./a.out < input03.txt | diff - output03.txt
@JSmevog1 →/workspaces/hw-group1-c-JSmevog1 (main) $ ./a.out < input04.txt | diff - output04.txt

```